

Einführung in Data Science

Python



Python

Inhalte

1. Überblick & Grundbegriffe
- ➡ 2. Einführung in Python (NumPy und Pandas)
3. Eindimensionale EDA und Visualisierungen
4. Visualisierungen und mehrdimensionale EDA
5. Mehrdimensionale EDA
6. Dimensionsreduktion: PCA
7. Dimensionsreduktion: MDS, Isomap
8. Clustering: Algorithmen
9. Clustering: Validierung
10. Probeklausur
11. Feature Engineering, datengetriebene Modelle
12. Datengetriebene Modelle
13. Zeitreihen

Überblick &
Grundlagen

Explorative
(Daten-)
Analyse (EDA)

Feature
Engineering &
Modellierung

Heute

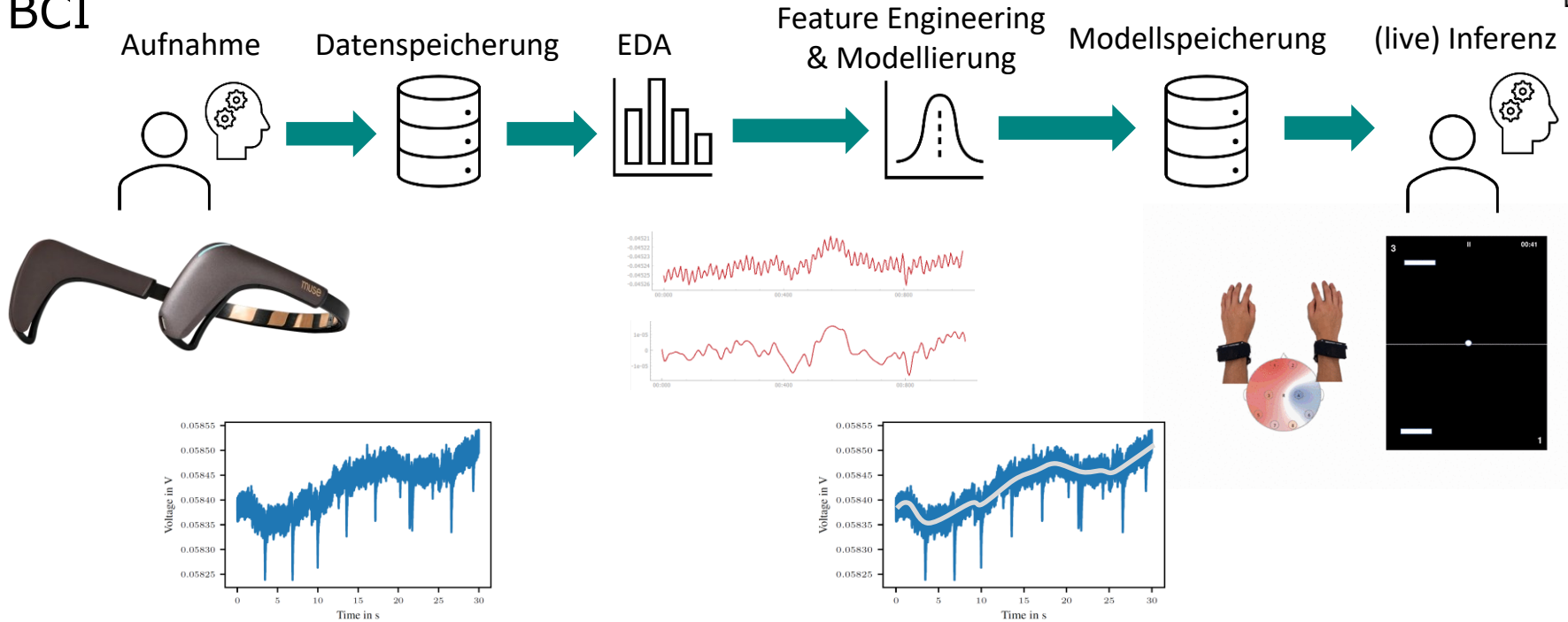
1. Wiederholung
2. Literatur und Lernunterlagen
3. Python (und übliche Data Science Bibliotheken)

1. Wiederholung

1. Wiederholung



BCI



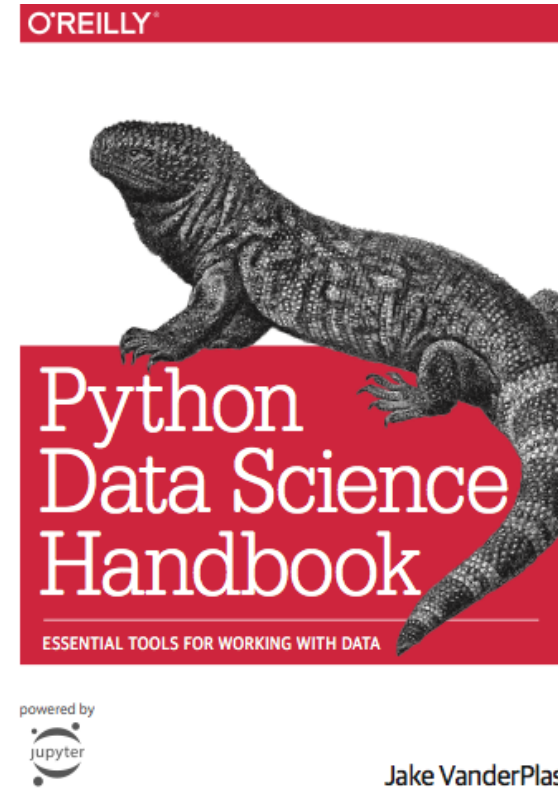
2. Literatur

2. Literatur

Python Data Science Handbook

Von Jake VanderPlas

- Anwendungsorientiert
- Übersicht über wichtigste Data Science Software-Bibliotheken für Python (NumPy, Pandas, matplotlib, scikit-learn)
- Autor ist Software Engineer bei Google Research; Entwickler von scikit-learn und Scipy
- Buch abrufbar:
<https://jakevdp.github.io/PythonDataScienceHandbook/>

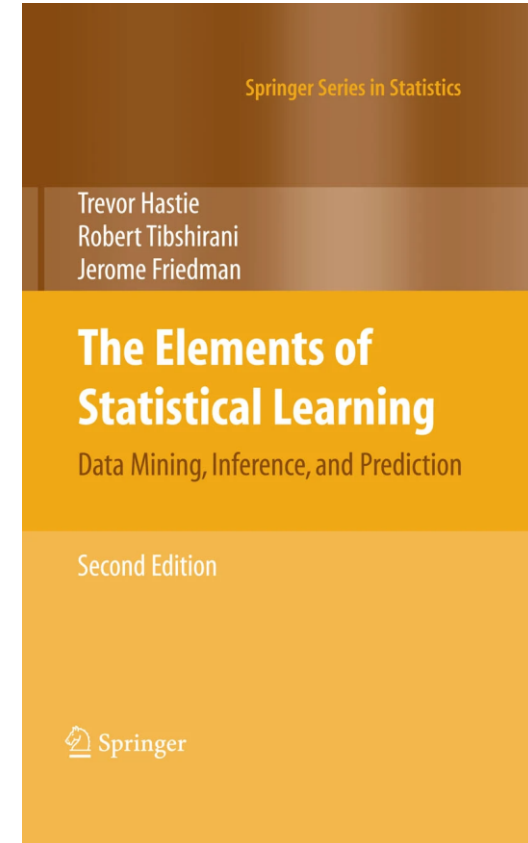


2. Literatur

The Elements of Statistical Learning

Von Trevor Hastie, Robert Tibshirani, Jerome Friedman

- Standardwerk für maschinelles Lernen
- (Mathematische) Grundlagen
- Erschienen: 2009
- Buch abrufbar:
<https://link.springer.com/book/10.1007/978-0-387-84858-7>

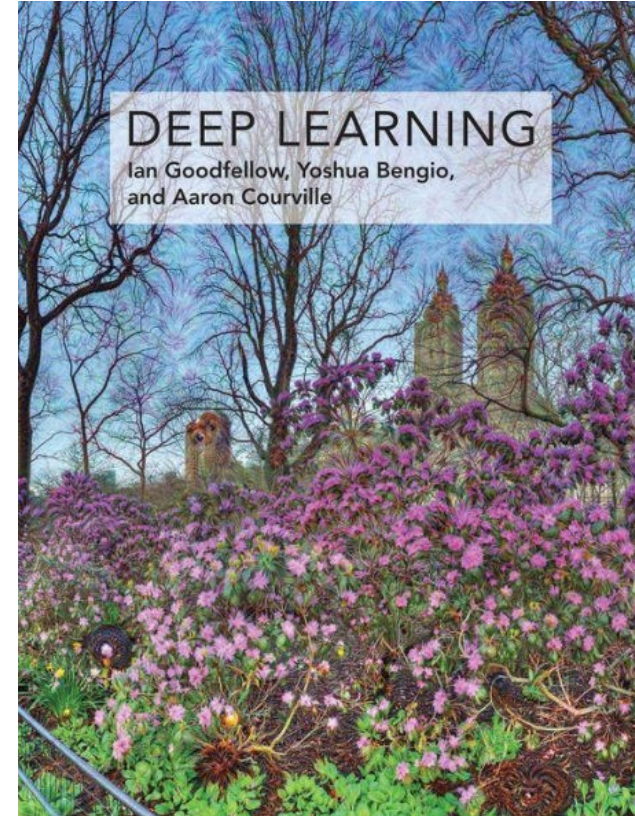


2. Literatur

Deep Learning

Von Ian Goodfellow, Yoshua Bengio, Aaron Courville

- Standardwerk für Deep Learning
 - Neuronale Netze
- (Mathematische) Grundlagen
- Erschienen: 2016
- Buch abrufbar:
<https://www.deeplearningbook.org/>



2. Literatur

Tutorials

- Kaggle: Tutorials zu Python, Data Science, Maschinellem Lernen, Deep Learning etc.: <https://www.kaggle.com/learn>
- Coursera: Machine Learning
<https://de.coursera.org/learn/machine-learning>
- Caltech: Machine Learning
<https://work.caltech.edu/telecourse.html#lectures>
- Scikit-Learn Documentation: <https://scikit-learn.org/stable/modules/classes.html>

3. Python

3. Python

- Einfache Syntax – leicht zu lernen und zu merken
 - Keine {} oder () Blöcke – nur Einrückung
- Effiziente Entwicklung – hohe Produktivität
- Interpretiert (nicht kompiliert)
- Garbage Collection
- Dynamische Typisierung
- Open Source (für jedes Problem gibt es ein Paket)
- Wie viele Python Pakete gibt es etwa?
 - 755.471 Pakete (Stand: 04.03.2026)
 - 614.386 Pakete (Stand: 06.03.2025)
 - 527.905 Pakete (Stand: 05.04.2024)



Quelle: pypi.org, python.org

3. Python

Beispiel

```
import math
```

Modul importieren

```
class Circle:
```

```
    def __init__(self, radius):  
        self.radius = radius
```

```
    def area(self):  
        area = math.pi * self.radius ** 2  
        return area
```

Klassendefinition

```
def get_radius(area):  
    radius = math.sqrt(area / math.pi)  
    return radius
```

Funktionsdefinition

Kommentar →

```
# Main program
```

```
if __name__ == '__main__':
```

Objektinstanziierung →

```
    circle = Circle(5)
```

Methodenaufruf →

```
    area = circle.area()
```

Funktionsaufruf →

```
    radius = get_radius(area)
```

Hauptteil des Programms
(main function)

3. Python

Beispiel

```
import math

class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        area = math.pi * self.radius ** 2
        return area

def get_radius(area):
    radius = math.sqrt(area / math.pi)
    return radius

# Main program
if __name__ == '__main__':
    circle = Circle(5)
    area = circle.area()
    radius = get_radius(area)
```

Wie können wir den Code besser lesbar machen?

- Type hints
- Docstrings
- Zeilenumbrüche

```
import math

class Circle:
    def __init__(self, radius: float):
        """
        Initialize new circle of specified radius.

        :param radius: Radius of circle as float.
        """
        self.radius = radius

    def area(self) -> float:
        """
        Return area of circle.

        :return: Area of circle as float.
        """
        area = math.pi * self.radius ** 2
        return area

    def get_radius(area: float) -> float:
        """
        Function to display the area of a given shape.

        :param area: Area of circle as float.
        :return: Radius of circle as float
        """
        radius = math.sqrt(area / math.pi)
        return radius

# Main program
if __name__ == '__main__':
    circle = Circle(5)
    area = circle.area()
    radius = get_radius(area)
```

3. Python

PyCharm (IDE)

- Code-Vervollständigung und -Analyse
 - Info, wenn Typ-Hinweise nicht mit dem Code übereinstimmen
 - Hervorheben von style violations
- Code-Navigation (zur Definition gehen, Verwendung finden)
- Projektverwaltung (und virtuelle Umgebungen)
- Plattformübergreifende Kompatibilität
- Integrierte Versionskontrolle
- Einfacher Debugger
- **Setup**: siehe Installationshandbuch
- PyCharm Professional kostenlos für Studierende



Quelle: Wikipedia

3. Python

Jupyter Notebooks

- Mischung aus Python-Code und Markdown-Zellen
 - Unterstützt Text, LaTeX-Gleichungen, Bilder und mehr
- Interaktive Berechnungen
 - Jeweils eine Zelle
 - Rückmeldung in Echtzeit
- Gut geeignet für Explorative Datenanalyse
 - Lädt die Daten nur einmal in den Arbeitsspeicher
 - Einfache Visualisierung
- **Setup:** siehe Installationshandbuch
- Integration in PyCharm Professional
- **Klausur** voraussichtlich in Jupyter-Lab



Quelle: Wikipedia

3. Python

Grundlegende Syntax

- Siehe Jupyter Notebook `topic02_python/basics.ipynb`
- Es gibt einige built-in Funktionen, z.B.

type() und isinstance()	Konvertierungsfunktionen	Funktionen und Methoden integrierter Datentypen
<pre>In [1]: type("Hello") Out[1]: str In [2]: type(123) Out[2]: int In [3]: isinstance("Hello", str) Out[3]: True In [4]: isinstance(123, str) Out[4]: False</pre>	<pre>In [1]: int("123") Out[1]: 123 In [2]: str(123) Out[2]: '123' In [3]: int(False) Out[3]: 0 In [4]: bool(1) Out[4]: True</pre>	<pre>In [1]: x = [0, 1, 2, 3] In [2]: len(x) Out[2]: 4 In [3]: x.append(4) In [4]: x Out[4]: [0, 1, 2, 3, 4] In [5]: x.extend([5, 6, 7]) In [6]: x Out[6]: [0, 1, 2, 3, 4, 5, 6, 7]</pre>

3. Python

Code in
topic02_python/exercise1.py

Übung

Lösen Sie die folgenden Aufgaben in Python:

1. Geben Sie „Hello, World!“ aus.
2. Erzeugen Sie eine Liste der Quadrate der Zahlen von 1 bis 10 und geben Sie diese aus.
3. Berechnen Sie mit `sum()` die Summe der geraden Zahlen zwischen 1 und 50.
4. Geben Sie ein Dreiecksmuster mit Sternchen (*) mit einer vorher festgelegten Anzahl von Zeilen aus, z.B. für $n = 5$:

```
*  
**  
***  
****  
*****
```

5. Berechnen Sie die Fakultät einer gegebenen Zahl, z. B. für $n = 5$:

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$$

Hinweis: Wenn Sie Python nicht installiert haben, können Sie einen Online-Python-Compiler wie <https://www.programiz.com/python-programming/online-compiler/> oder <https://www.online-python.com/> verwenden.

3. Python

Wichtige Strukturen – Dictionaries

- Nützlich zum Speichern von (unstrukturierten) Informationen
- Ähnlich dem JSON-Format

```
x = {'german': 'Hallo', 'english': 'Hello', 'spanish': 'Hola'}
```

x['german']	'Hallo'
x['catalan']	KeyError: 'catalan'
x.get('german', 'Hey')	'Hallo'
x.get('catalan', 'Hey')	'Hey'
x.keys()	['german', 'english', 'spanish']
x.values()	['Hallo', 'Hello', 'Hola']
x.items()	[('german', 'Hallo'), ('english', 'Hello'), ('spanish', 'Hola')]

3. Python

Wichtige Strukturen – Funktionen

- Input: arguments (args) und keyword arguments (kwargs)
- Output: return value (Standard: None)
- Funktionsblock durch Einrückung gekennzeichnet
- Kann als Wert einer Variablen zugewiesen werden

```
function1 = summation
sum1 = function1(5, 6)
print(sum1)
>>> 11
```

- Anonyme Funktionen durch Lambda-Ausdrücke

```
function2 = lambda a, b: a + b
sum1 = function2(5, 6)
print(sum1)
>>> 11
```

Funktionsname keyword arguments
arguments return value type hint

```
def summation(a: int, b: int, c: int = 0) -> int:
    """
    Sums two (or three) integers.

    :param a: First integer.
    :param b: Second integer.
    :param c: Third integer (optional). Defaults to 0.
    :return: Sum of the integers.
    """
    return a + b + c

if __name__ == '__main__':
    sum1 = summation(5, 6)
    sum2 = summation(5, 6, c=7)

    print(f"sum1={sum1}, sum2={sum2}")
```

Output

```
>>> sum1=11, sum2=18
```

3. Python

Code in
topic02_python/exercise2.py

Übung

1. Schreiben Sie eine Funktion, die als Eingabe zwei ganze Zahlen und einen Operator als String („+“, „-“, „*“, „/\") annimmt und die gewünschte Berechnung durchführt.

```
def calculator(a: int, b: int, operator: str) -> float:
```

2. Erstellen Sie eine Funktion, die die durchschnittliche Punktzahl basierend auf einem Dictionary mit einzelnen Punkten berechnet, z.B. {"Statistik": 85, "Analysis": 78, "Data Science": 92}. Berechnen Sie die Durchschnittsnote und geben Sie diese zurück

```
def average_grade(grades: dict) -> float:
```

Hinweis: Wenn Sie Python nicht installiert haben, können Sie einen Online-Python-Compiler wie <https://www.programiz.com/python-programming/online-compiler/> oder <https://www.online-python.com/> verwenden.

3. Python

Wichtige Strukturen – Klassen

- Eine Klassendefinition kann verschiedene Komponenten enthalten
 - Klassenattribute
 - Instanz-Attribute
 - Methoden
 - Statische Methoden
- Die Methode `__init__()` definiert, was passiert, wenn ein Objekt initialisiert wird

```
class Rectangle:
    version = "1.0.0"

    def __init__(self, height, width):
        self.height = height
        self.width = width

    def area(self):
        area = self.height * self.width
        return area

    def get_description(self):
        description = ("The area of a rectangle is the "
                      "product of its height and width")

        return description

if __name__ == '__main__':
    rect = Rectangle(3, 5)
    area = rect.area()
    description = rect.get_description()
    print(description)
    print(f"The rectangle's height is {rect.height}, its "
          f"width is {rect.width}, so its area is {area}")
```

Output

```
>>> The area of a rectangle is the product of its height and width
>>> The rectangle's height is 3, its width is 5, so its area is 15
```

3. Python

Wichtige Strukturen – Klassen

- Eine Klassendefinition kann verschiedene Komponenten enthalten
 - Klassenattribute
 - Instanz-Attribute
 - Methoden
 - Statische Methoden
- Die Methode `__init__()` definiert, was passiert, wenn ein Objekt initialisiert wird

```
class Rectangle:
    version = "1.0.0"

    def __init__(self, height, width):
        self.height = height
        self.width = width

    def area(self):
        area = self.height * self.width
        return area

    @staticmethod
    def get_description():
        description = ("The area of a rectangle is the "
                      "product of its height and width")

        return description

if __name__ == '__main__':
    rect = Rectangle(3, 5)
    area = rect.area()
    description = rect.get_description()
    print(description)
    print(f"The rectangle's height is {rect.height}, its "
          f"width is {rect.width}, so its area is {area}")
```

Output

```
>>> The area of a rectangle is the product of its height and width
>>> The rectangle's height is 3, its width is 5, so its area is 15
```

3. Python

Wichtige Strukturen – Vererbung

- In Python gibt es keine Interfaces
- Stattdessen können wir *abstract base classes* definieren, die von ABC erben
- Klassen, die von *abstract base classes* erben, müssen alle abstrakten Methoden implementieren
- Um eine Methode einer Elternklasse zu überschreiben, definieren wir sie in der Kindklasse
- Um eine Methode einer übergeordneten Klasse aufzurufen und etwas hinzuzufügen, rufen wir sie in der untergeordneten Klasse auf.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Rectangle(Shape):
    def __init__(self, height, width):
        self.height = height
        self.width = width

    def area(self):
        area = self.height * self.width
        return area

class Square(Rectangle):
    def __init__(self, side_length):
        super().__init__(side_length, side_length)

if __name__ == '__main__':
    square = Square(5)
    area = square.area()
    print(f"The square's height is {square.height}, its "
          f"width is {square.width}, so its area is {area}")
```

Output

```
>>> The square's height is 5, its width is 5, so its area is 25
```


3. Python

Code in
topic02_python/exercise3.py

Übung

1. Erstellen Sie eine Klasse BankAccount mit Attributen für den Namen und den Saldo des Kontoinhabers. Implementieren Sie Methoden zum Einzahlen und Abheben. Stellen Sie sicher, dass nur Geld abgehoben werden kann, so dass der Saldo nicht negativ sein kann. Erstellen Sie Instanzen dieser Klasse und führen Sie Transaktionen durch.
2. Erstellen Sie eine Klasse Student mit den Attributen Name, Alter und Noten hat (als Dictionary, z.B. {„Statistics“: 85, „Math“: 78, „IAI“: 92}). Implementieren Sie eine Methode zum Hinzufügen von Noten zum Dictionary. Erstellen Sie Instanzen der Klasse und fügen Sie Noten hinzu.

Hinweis: Wenn Sie Python nicht installiert haben, können Sie einen Online-Python-Compiler wie <https://www.programiz.com/python-programming/online-compiler/> oder <https://www.online-python.com/> verwenden.

3. Python

Wichtige Strukturen – Module

- Ein Modul ist eine Python-Datei (.py), die Definitionen und Anweisungen enthält
- Wir können Module verwenden, um Code in verschiedene Dateien aufzuteilen und ihn sauber zu halten
- Module werden mit dem import Statement importiert

Output:

```
>>> The square's is Green and its area is 25
```

```
# rectangle.py
# -----
class Rectangle:
    def __init__(self, height, width):
        self.height = height
        self.width = width

    def area(self):
        area = self.height * self.width
        return area

if __name__ == '__main__':
    rect = Rectangle(3, 5)
    area = rect.area()
    print(f"The rectangle's height is {rect.height}, its "
          f"width is {rect.width}, so its area is {area}")
```

```
# square.py
# -----
import rectangle

class Square(rectangle.Rectangle):
    def __init__(self, side_length):
        super().__init__(side_length, side_length)

if __name__ == '__main__':
    square = Square(5)
    area = square.area()
    print(f"The square's height is {square.height}, its "
          f"width is {square.width}, so its area is {area}")
```

3. Python

Wichtige Strukturen – Module

- Ein Modul ist eine Python-Datei (.py), die Definitionen und Anweisungen enthält
- Wir können Module verwenden, um Code in verschiedene Dateien aufzuteilen und ihn sauber zu halten
- Module werden mit dem import Statement importiert
- Einzelne Definitionen können mit “from ... import ...” importiert werden
- Bad practice: Wildcards verwenden, z. B. from rectangle import *

```
# rectangle.py
# -----
class Rectangle:
    def __init__(self, height, width):
        self.height = height
        self.width = width

    def area(self):
        area = self.height * self.width
        return area

if __name__ == '__main__':
    rect = Rectangle(3, 5)
    area = rect.area()
    print(f"The rectangle's height is {rect.height}, its "
          f"width is {rect.width}, so its area is {area}")
```

```
# square.py
# -----
from rectangle import Rectangle

class Square(Rectangle):
    def __init__(self, side_length):
        super().__init__(side_length, side_length)

if __name__ == '__main__':
    square = Square(5)
    area = square.area()
    print(f"The square's height is {square.height}, its "
          f"width is {square.width}, so its area is {area}")
```

3. Python

Wichtige Strukturen – Pakete

- In größeren Projekten können wir mehrere Module zu Paketen zusammenfassen
- Ein Paket ist ein Ordner mit `__init__.py` Datei

```
project
├── __init__.py
├── colored_shape.py
├── shapes
│   ├── __init__.py
│   ├── shape.py
│   ├── rectangle.py
│   └── square.py
├── colors
│   ├── __init__.py
│   ├── red.py
│   ├── green.py
│   └── blue.py
```

```
# colored_shape.py
# -----
from colors.green import Green
from shapes.square import Square

class ColoredShape:
    def __init__(self, color, shape):
        self.color = color
        self.shape = shape

    def area(self):
        return self.shape.area()

    def get_color(self):
        return self.color.get_color()

if __name__ == '__main__':
    square = Square(5)
    green = Green()
    green_square = ColoredShape(green, square)

    print(f"The square's is {green_square.get_color()} "
          f"and its area is {green_square.area()}")
```

Output

```
>>> The square's is Green and its area is 25
```

3. Python

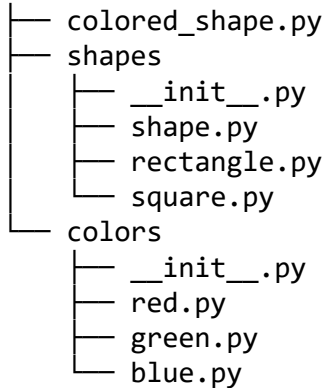
```
# shapes/__init__.py
# -----
from shape import Shape
from rectangle import Rectangle
from square import Square
```

```
# colors/__init__.py
# -----
from red import Red
from green import Green
from blue import Blue
```

Wichtige Strukturen – Pakete

- Wir können Definitionen direkt in der `__init__.py` Datei importieren und sie so direkt zugänglich machen

Alternativ



```
from colors.green import Green
from shapes.square import Square
```

```
# colored_shape.py
# -----
from colors import Green
from shapes import Square

class ColoredShape:
    def __init__(self, color, shape):
        self.color = color
        self.shape = shape

    def area(self):
        return self.shape.area()

    def get_color(self):
        return self.color.get_color()

if __name__ == '__main__':
    square = Square(5)
    green = Green()
    green_square = ColoredShape(green, square)

    print(f"The square's is {green_square.get_color()} "
          f"and its area is {green_square.area()}")
```

3. Python

Wichtige Strukturen – Pakete

- Neben unseren eigenen (internen) Paketen verwenden wir häufig auch (externe) Pakete von Dritten
- Übliche Pakete sind
 - NumPy: effiziente numerische Berechnungen
 - SciPy: wissenschaftliche Berechnungen (basierend auf NumPy)
 - Pandas: Datenmanipulation und -analyse
 - Matplotlib: statische, animierte und interaktive Diagramme
 - Scikit-learn: Bibliothek für maschinelles Lernen
 -
 - TensorFlow: Bibliothek für tiefes Lernen (Google)
 - PyTorch: Bibliothek für tiefes Lernen (Meta)
 - XGBoost: skalierbares und genaues Gradient Boosting



Quelle: Wikidata

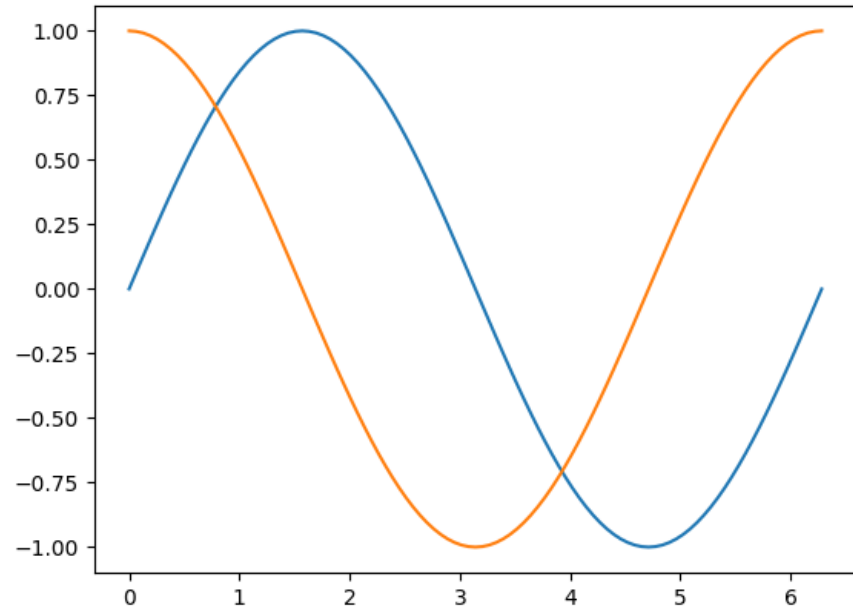
3. Python

Wichtige Strukturen – Pakete

```
import numpy as np
import matplotlib.pyplot as plt

if __name__ == '__main__':
    x = np.linspace(0, 2 * np.pi, 101)
    y = np.sin(x)
    z = np.cos(x)

    plt.plot(x, y)
    plt.plot(x, z)
    plt.show()
```



3. Python

Übung

Entwickeln Sie eine Filmdatenbank

1. Erstellen Sie ein Modul `movie.py` mit einer Klasse `Movie`, die Attribute wie Titel, Jahr und Genre enthält. Implementieren Sie außerdem eine Methode zur Anzeige dieser Informationen.
2. Erstellen Sie ein Modul `catalog.py` mit einer Klasse `Catalog`, die eine Liste von Filmen enthält. Implementieren Sie Methoden zum Hinzufügen von Filmen und zur Anzeige des gesamten Katalogs.
3. Entwickeln Sie ein Modul `main.py`, in dem Sie Instanzen von Filmen erstellen, diese dem Katalog hinzufügen und den gesamten Katalog anzeigen.

Hinweis: Wenn Sie Python nicht installiert haben, können Sie einen Online-Python-Compiler wie <https://www.programiz.com/python-programming/online-compiler/> oder <https://www.online-python.com/> verwenden.

3. Python

Ressourcen

- Offizielle Website: <https://www.python.org/>
- Dokumentation: <https://docs.python.org/3/>
- Beginner's Guide: <https://wiki.python.org/moin/BeginnersGuide>
- Tutorials:
 - python.org: <https://docs.python.org/3/tutorial/index.html>
 - Kaggle: <https://www.kaggle.com/learn/python>
 - W3 Schools: <https://www.w3schools.com/python/>
- PowerPoint slides (von Chaiporn Jaokaew und James Brucker, 2020):
<https://www.slideserve.com/anthonyahmed/introduction-to-the-python-language-powerpoint-ppt-presentation>

Überblick & Grundbegriffe

Rückblick

- Wie werden Codeblöcke in Python gekennzeichnet?
- Welche Vorteile hat Python?
- Wie kann man Python-Code lesbar machen?
- Was ist der Unterschied zwischen einer Liste und einem Tupel in Python?
- Welche zwei Arten von Argumenten gibt es für Funktionen?
- Was ist der Unterschied zwischen Klassenattributen und Instanzattributen?
- Was ist die Verwendung von ABC (abstract base class) in Python?
- Was ist ein Python-Modul?
- Warum verwenden wir Module und Pakete?

Überblick & Grundbegriffe

Rückblick

- Wird Python-Code kompiliert oder interpretiert?
- Sind Type Hints notwendig?
- Was sind Dictionaries in Python?
- Was ist in Python der Unterschied zwischen `[1, 2, 3]`, `(1, 2, 3)` und `{1, 2, 3}`?
- Wofür wird der Lambda-Operator verwendet?
- Was ist der Unterschied zwischen einer Methode und einer statischen Methode in einer Python-Klasse?
- Was ist ein Python-Paket?
- Wie können Module in Python importiert werden?
- Wie kann eine einzelne Definition in Python importiert werden?